OS LAB - Producer-Consumer with Atomic Primitives

Name: Aditya Kumar
Registration no: AP24110010333
BTech CSE – C

Implement the bounded-buffer Producer-Consumer problem in C using pthreads. The buffer has size N = 5, with 3 producers and 3 consumers, each handling 10 items. A shared counter tracks the number of items in the buffer, and a lock variable protects the critical section. The lock is built only from the atomic primitives test_and_set and compare_and_swap (no pthread_mutex_t, sem_t or other language-level locks). The program has two builds selected by USE_CAS (0 = test_and_set, 1 = compare_and_swap). The counter invariant 0 ≤ counter ≤ BUFFER_SIZE is checked with assert() inside the critical section, and the final counter must be 0.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdatomic.h>
#include <stdbool.h>
#include <unistd.h>
#include <assert.h>
#include <sched.h>

#ifndef USE_CAS
#define USE_CAS 0   /* 0 = test_and_set, 1 = compare_and_swap */
#endif

#ifndef USE_LOCK
#define USE_LOCK 1  /* 0 = remove acquire/release (race experiment) */
#endif

#define BUFFER_SIZE 5

#ifndef NUM_PRODUCERS
#define NUM_PRODUCERS 3
#endif

#ifndef NUM_CONSUMERS
#define NUM_CONSUMERS 3
#endif

#define ITEMS_PER_THREAD 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int counter = 0;              // shared counter to verify full / empty
atomic_bool lock = false;     // hardware lock variable
```

```
// Statistics (protected by the lock, except where noted)
int total_produced = 0, total_consumed = 0;
int max_counter_seen = 0;
atomic_int assertion_failures = 0;

// ---------- Atomic primitives ----------
bool test_and_set(atomic_bool *target) {
    return atomic_exchange(target, true);   // returns OLD value
}

bool compare_and_swap(atomic_bool *target, bool expected, bool new_val) {
    return atomic_compare_exchange_strong(target, &expected, new_val);
}

// ---------- Lock / Unlock ----------
void acquire(atomic_bool *l) {
#if USE_CAS
    while (!compare_and_swap(l, false, true)) {
        // CAS failed -> someone else holds the lock -> retry
    }
#else
    while (test_and_set(l)) {
        // busy-wait: spin until lock is free
    }
#endif
}

void release(atomic_bool *l) {
    atomic_store(l, false);
}

#if USE_LOCK
#define LOCK()   acquire(&lock)
#define UNLOCK() release(&lock)
#else
#define LOCK()
#define UNLOCK()
#endif

// ---------- Invariant check ----------
// With the lock enabled a failed check aborts via assert().
// In the no-lock experiment we count failures instead so the run can finish.
static void check_invariant(const char *who, int id) {
#if USE_LOCK
    assert(counter >= 0 && counter <= BUFFER_SIZE);
    (void)who; (void)id;
#else
    if (!(counter >= 0 && counter <= BUFFER_SIZE)) {
        atomic_fetch_add(&assertion_failures, 1);
    }
    (void)who; (void)id;
#endif
}

// ---------- Producer / Consumer ----------
void *producer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < ITEMS_PER_THREAD; i++) {
        int item = id * 100 + i;
        bool done = false;
        while (!done) {
```

```
            LOCK();
            if (counter < BUFFER_SIZE) {          // buffer not full
                buffer[in] = item;
                in = (in + 1) % BUFFER_SIZE;
                counter++;
                total_produced++;
                if (counter > max_counter_seen) max_counter_seen = counter;
                check_invariant("Producer", id);
                printf("Producer %d produced item %3d | counter = %d\n", id, item, counter);
                done = true;
            }
            UNLOCK();
            if (!done) sched_yield();             // buffer full: retry
        }
    }
    return NULL;
}

void *consumer(void *arg) {
    int id = *(int *)arg;
    for (int i = 0; i < ITEMS_PER_THREAD; i++) {
        bool done = false;
        while (!done) {
            LOCK();
            if (counter > 0) {                    // buffer not empty
                int item = buffer[out];
                out = (out + 1) % BUFFER_SIZE;
                counter--;
                total_consumed++;
                check_invariant("Consumer", id);
                printf("Consumer %d consumed item %3d | counter = %d\n", id, item, counter);
                done = true;
            }
            UNLOCK();
            if (!done) sched_yield();             // buffer empty: retry
        }
    }
    return NULL;
}

int main(void) {
    pthread_t prod[NUM_PRODUCERS], cons[NUM_CONSUMERS];
    int pid[NUM_PRODUCERS], cid[NUM_CONSUMERS];

    printf("Lock type : %s\n", USE_CAS ? "compare_and_swap (USE_CAS 1)" : "test_and_set
(USE_CAS 0)");
    printf("Lock used : %s\n", USE_LOCK ? "yes" : "NO (race experiment)");
    printf("Config    : %d producer(s), %d consumer(s), %d items each, buffer size %d\n\n",
           NUM_PRODUCERS, NUM_CONSUMERS, ITEMS_PER_THREAD, BUFFER_SIZE);

    for (int i = 0; i < NUM_PRODUCERS; i++) {
        pid[i] = i + 1;
        pthread_create(&prod[i], NULL, producer, &pid[i]);
    }
    for (int i = 0; i < NUM_CONSUMERS; i++) {
        cid[i] = i + 1;
        pthread_create(&cons[i], NULL, consumer, &cid[i]);
    }

    for (int i = 0; i < NUM_PRODUCERS; i++) pthread_join(prod[i], NULL);
    for (int i = 0; i < NUM_CONSUMERS; i++) pthread_join(cons[i], NULL);
```

```
    printf("\nTotal produced = %d\n", total_produced);
    printf("Total consumed = %d\n", total_consumed);
    printf("Max counter observed = %d (limit %d)\n", max_counter_seen, BUFFER_SIZE);
#if !USE_LOCK
    printf("Invariant violations detected = %d\n", atomic_load(&assertion_failures));
#endif
    printf("\nFinal counter = %d (expected 0)\n", counter);
    return 0;
}
```

OUTPUT:

For case-1: USE_CAS 0 (test_and_set), 3 producers, 3 consumers

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ gcc -pthread -DUSE_CAS=0 producer_consumer.c -o pc_tas
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./pc_tas
Lock type : test_and_set (USE_CAS 0)
Lock used : yes
Config    : 3 producer(s), 3 consumer(s), 10 items each, buffer size 5

Producer 1 produced item 100 | counter = 1
Producer 1 produced item 101 | counter = 2
Producer 1 produced item 102 | counter = 3
Producer 1 produced item 103 | counter = 4
Producer 1 produced item 104 | counter = 5
Consumer 1 consumed item 100 | counter = 4
Consumer 1 consumed item 101 | counter = 3
Consumer 1 consumed item 102 | counter = 2
Consumer 1 consumed item 103 | counter = 1
Consumer 1 consumed item 104 | counter = 0
Producer 1 produced item 105 | counter = 1
Producer 1 produced item 106 | counter = 2
Producer 1 produced item 107 | counter = 3
Producer 1 produced item 108 | counter = 4
Producer 1 produced item 109 | counter = 5
Consumer 1 consumed item 105 | counter = 4
Consumer 1 consumed item 106 | counter = 3
Consumer 1 consumed item 107 | counter = 2
Consumer 1 consumed item 108 | counter = 1
Consumer 1 consumed item 109 | counter = 0
Producer 3 produced item 300 | counter = 1
Producer 3 produced item 301 | counter = 2
Producer 3 produced item 302 | counter = 3
Producer 3 produced item 303 | counter = 4
Producer 3 produced item 304 | counter = 5
Consumer 2 consumed item 300 | counter = 4
Consumer 2 consumed item 301 | counter = 3
Consumer 2 consumed item 302 | counter = 2
Producer 2 produced item 200 | counter = 3
Producer 2 produced item 201 | counter = 4
Producer 2 produced item 202 | counter = 5
Consumer 2 consumed item 303 | counter = 4
Consumer 2 consumed item 304 | counter = 3
Consumer 2 consumed item 200 | counter = 2
Consumer 2 consumed item 201 | counter = 1
Consumer 2 consumed item 202 | counter = 0
Producer 3 produced item 305 | counter = 1
Producer 3 produced item 306 | counter = 2
Producer 3 produced item 307 | counter = 3
Producer 3 produced item 308 | counter = 4
```

```
Producer 3 produced item 309 | counter = 5
Consumer 3 consumed item 305 | counter = 4
Consumer 3 consumed item 306 | counter = 3
Consumer 3 consumed item 307 | counter = 2
Consumer 3 consumed item 308 | counter = 1
Consumer 3 consumed item 309 | counter = 0
Producer 2 produced item 203 | counter = 1
Producer 2 produced item 204 | counter = 2
Producer 2 produced item 205 | counter = 3
Producer 2 produced item 206 | counter = 4
Producer 2 produced item 207 | counter = 5
Consumer 2 consumed item 203 | counter = 4
Consumer 2 consumed item 204 | counter = 3
Producer 2 produced item 208 | counter = 4
Producer 2 produced item 209 | counter = 5
Consumer 3 consumed item 205 | counter = 4
Consumer 3 consumed item 206 | counter = 3
Consumer 3 consumed item 207 | counter = 2
Consumer 3 consumed item 208 | counter = 1
Consumer 3 consumed item 209 | counter = 0

Total produced = 30
Total consumed = 30
Max counter observed = 5 (limit 5)

Final counter = 0 (expected 0)
```

For case-2: USE_CAS 1 (compare_and_swap), 3 producers, 3 consumers

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ gcc -pthread -DUSE_CAS=1 producer_consumer.c -o pc_cas
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./pc_cas
Lock type : compare_and_swap (USE_CAS 1)
Lock used : yes
Config    : 3 producer(s), 3 consumer(s), 10 items each, buffer size 5

Producer 2 produced item 200 | counter = 1
Producer 2 produced item 201 | counter = 2
Producer 2 produced item 202 | counter = 3
Producer 2 produced item 203 | counter = 4
Producer 2 produced item 204 | counter = 5
Consumer 1 consumed item 200 | counter = 4
Consumer 1 consumed item 201 | counter = 3
Consumer 1 consumed item 202 | counter = 2
Consumer 1 consumed item 203 | counter = 1
Consumer 1 consumed item 204 | counter = 0
Producer 1 produced item 100 | counter = 1
Producer 1 produced item 101 | counter = 2
Producer 1 produced item 102 | counter = 3
Producer 1 produced item 103 | counter = 4
Producer 1 produced item 104 | counter = 5
Consumer 1 consumed item 100 | counter = 4
Consumer 1 consumed item 101 | counter = 3
Consumer 1 consumed item 102 | counter = 2
Consumer 1 consumed item 103 | counter = 1
Consumer 1 consumed item 104 | counter = 0
Producer 1 produced item 105 | counter = 1
Producer 1 produced item 106 | counter = 2
Producer 1 produced item 107 | counter = 3
Producer 1 produced item 108 | counter = 4
Producer 1 produced item 109 | counter = 5
Consumer 2 consumed item 105 | counter = 4
```

```
Consumer 2 consumed item 106 | counter = 3
Consumer 2 consumed item 107 | counter = 2
Consumer 2 consumed item 108 | counter = 1
Consumer 2 consumed item 109 | counter = 0
Producer 2 produced item 205 | counter = 1
Producer 2 produced item 206 | counter = 2
Producer 2 produced item 207 | counter = 3
Producer 2 produced item 208 | counter = 4
Producer 2 produced item 209 | counter = 5
Consumer 3 consumed item 205 | counter = 4
Consumer 3 consumed item 206 | counter = 3
Consumer 3 consumed item 207 | counter = 2
Consumer 3 consumed item 208 | counter = 1
Consumer 3 consumed item 209 | counter = 0
Producer 3 produced item 300 | counter = 1
Producer 3 produced item 301 | counter = 2
Producer 3 produced item 302 | counter = 3
Producer 3 produced item 303 | counter = 4
Producer 3 produced item 304 | counter = 5
Consumer 2 consumed item 300 | counter = 4
Consumer 2 consumed item 301 | counter = 3
Consumer 2 consumed item 302 | counter = 2
Consumer 2 consumed item 303 | counter = 1
Consumer 2 consumed item 304 | counter = 0
Producer 3 produced item 305 | counter = 1
Producer 3 produced item 306 | counter = 2
Producer 3 produced item 307 | counter = 3
Producer 3 produced item 308 | counter = 4
Producer 3 produced item 309 | counter = 5
Consumer 3 consumed item 305 | counter = 4
Consumer 3 consumed item 306 | counter = 3
Consumer 3 consumed item 307 | counter = 2
Consumer 3 consumed item 308 | counter = 1
Consumer 3 consumed item 309 | counter = 0

Total produced = 30
Total consumed = 30
Max counter observed = 5 (limit 5)

Final counter = 0 (expected 0)
```

For case-3: no acquire/release, 1 producer, 1 consumer (typical run)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ gcc -pthread -DUSE_LOCK=0 -DNUM_PRODUCERS=1 -
DNUM_CONSUMERS=1 producer_consumer.c -o pc_nolock
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./pc_nolock
Lock type : test_and_set (USE_CAS 0)
Lock used : NO (race experiment)
Config    : 1 producer(s), 1 consumer(s), 10 items each, buffer size 5

Producer 1 produced item 100 | counter = 1
Producer 1 produced item 101 | counter = 1
...
Consumer 1 consumed item 108 | counter = 0
Consumer 1 consumed item 109 | counter = 0

Total produced = 10
Total consumed = 10
Max counter observed = 5 (limit 5)
Invariant violations detected = 0

Final counter = 0 (expected 0)
```

For case-3: same build, a run where the race corrupted the counter

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./pc_nolock
Lock type : test_and_set (USE_CAS 0)
Lock used : NO (race experiment)
Config    : 1 producer(s), 1 consumer(s), 10 items each, buffer size 5

Producer 1 produced item 100 | counter = 1
Consumer 1 consumed item 100 | counter = 0
...
Consumer 1 consumed item 108 | counter = 1
Consumer 1 consumed item 109 | counter = 1

Total produced = 10
Total consumed = 10
Max counter observed = 2 (limit 5)
Invariant violations detected = 0

Final counter = 1 (expected 0)
```

Both locked builds produced 30 items and consumed 30 items, the counter never left the range 0 to 5 (the assertion never fired), and the final counter was 0.

REPORT:

1. Why must test_and_set be atomic?

test_and_set has to read the old value and write true as one indivisible step. If it were two separate steps (read the lock, then set it), two threads could both read false before either one wrote true. Both would then believe they own the lock and enter the critical section together, so mutual exclusion would be lost. Being atomic guarantees that exactly one thread sees the old value false; every other thread sees true and keeps spinning.

2. TAS vs CAS: memory writes on the failure path

	test_and_set (atomic_exchange)	**compare_and_swap**
Lock is free	Reads false, writes true: acquired	Compare succeeds, writes true: acquired
Lock is held (failure path)	Still writes true (1 write per spin)	Compare fails, no write (0 writes per spin)
Effect while spinning	Every spin dirties the cache line	Spinning can stay read-only

A failed test_and_set (atomic_exchange) still stores true into the lock, which was already true, so every spin is a write. A failed compare_and_swap sees that the lock is not equal to the expected false and leaves memory unchanged, so it makes no write. The difference matters on cache-coherent multiprocessors, where repeated writes keep invalidating the lock's cache line in other cores. In practice, some CPUs (x86 lock cmpxchg, for example) still take the line exclusively even when the compare fails, so the gain is smaller than the C semantics suggest. Spinning on a plain read first (test-and-test-and-set) avoids most of this traffic.

3. Does this lock guarantee bounded waiting?

No. Both spinlocks give mutual exclusion, but nothing decides which waiting thread gets the lock next. A thread can lose the race every time the lock is released and so starve, while the other threads keep reacquiring it.

Bounded waiting needs an explicit ordering, such as a ticket lock or the bounded-waiting test_and_set algorithm that uses a waiting[] array and hands the lock to the next waiting thread on release.

4. 1 producer, 1 consumer, acquire/release removed: does the assertion fire?

In my runs it did not. I ran the no-lock build several hundred times (at -O0 and -O2) and the check 0 ≤ counter ≤ 5 never failed. The wrong results showed up elsewhere, as in the second case-3 log: occasionally the final counter was 1 instead of 0, and the maximum counter seen was sometimes lower than in a clean run.

The reason is that counter++ and counter-- are each a read, a modify and a write. When the producer's increment and the consumer's decrement overlap, one update overwrites the other (a lost update), so the counter drifts away from the true number of items. With only two threads the drift is small, so the value stays inside 0 to 5 and the range assertion has nothing to catch, yet the counter no longer matches the buffer contents. The assertion is therefore a weak detector: with the lock it is guaranteed to hold, and without the lock a passing assertion proves nothing. The final-counter check (expected 0) is what exposed the race. The program also has a data race on in, out and the buffer, which is undefined behaviour in C11. In one batch of runs with 3 producers and 3 consumers and no lock, a run stalled and never finished, which is another symptom of the same corruption.